

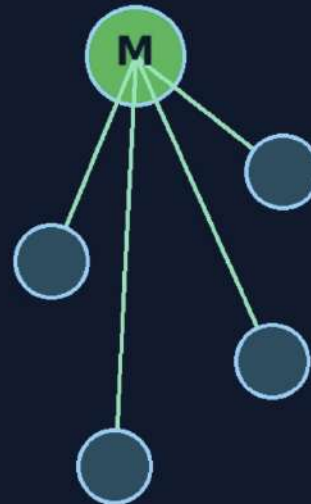
Spotify: Personalized AI Memory System

SPOTIFY

Personalized AI Memory System

Internal Product Dossier

Personalization / Retention
Personalized AI experiences



2. Why This Release Is Time-Critical

The immediate pressure is the growing gap between one-session intelligence and sustained personalization. As Spotify introduces more conversational and generative touchpoints, isolated session context will make experiences feel repetitive even when the underlying catalog and recommendation capabilities are strong. Each additional surface increases the cost of fragmented memory and the risk that one interaction contradicts another.

Retention pressure also changes the standard for quality. A useful first response is no longer enough; people expect the system to adapt across repeated visits, preserve explicit choices, and avoid resurfacing preferences they have corrected. If Spotify delays a shared memory capability, individual teams are likely to create local stores, inconsistent schemas, and duplicated retrieval logic. That path raises privacy risk, slows experimentation, and makes deletion or policy enforcement difficult to execute reliably.

Leadership therefore expects a short, controlled release cycle focused on a measurable memory lifecycle: capture, normalize, graph write, retrieval, context composition, response, feedback, and deletion. The pilot must show low-latency retrieval, reliable provenance, policy-aware context injection, a visible fallback when memory is unavailable, and measurable uplift against a memory-disabled baseline.

Leadership deadline

Deliver a reviewable pilot in six weeks, followed by a two-week controlled validation window. Expansion is gated on privacy approval, reliability thresholds, and demonstrated improvement in memory relevance—not on feature count.

3. Leadership Expectations

Outcome Area	Expectation	Why It Matters
Personal continuity	Carry forward durable listening, playlist, mood, podcast, and interaction preferences only when relevant to the current intent.	Continuity is the product value; indiscriminate recall would damage trust.
Memory precision	Prefer a small set of high-confidence memories over a large context dump, with source, time, and confidence attached.	Irrelevant context increases latency and can distort generation.
Graph integrity	Represent people, content, preferences, episodes, and relationships in a versioned temporal graph.	Graph structure supports explainable relationships, corrections, and change over time.
User agency	Support inspection, correction, opt-out, and deletion flows with propagation across graph and vector indexes.	Personalization must remain controllable and reversible.
Shared access	Expose memory read/write capabilities through authenticated MCP tools and stable APIs.	A common contract prevents surface-specific memory silos.
Operational reliability	Meet latency, availability, observability, and fallback thresholds before wider integration.	Memory cannot become a single point of failure for core AI journeys.
Measured impact	Compare memory-enabled and memory-disabled journeys using relevance, correction, repeat-use, and safety metrics.	Leadership needs evidence that memory creates durable value without increasing risk.

4. Leadership Meeting Transcript

The following record captures the release-definition discussion across product, engineering, data, AI, privacy, security, operations, and quality leadership. Designations are used throughout.

Chief Product Officer: Our AI experiences cannot feel personal if every interaction starts from zero. The goal is not simply to remember more. It is to preserve the few facts and patterns that make the next experience meaningfully better, while making that behavior understandable and controllable.

Head of Personalization: We already have strong behavioral signals, but an AI memory layer has a different job. Recommendation features optimize from aggregate patterns; this system needs conversational and episodic continuity, explicit corrections, and a representation of why a preference was retained.

Product Manager: The first release should focus on three journeys: refining a listening request over multiple interactions, carrying durable playlist preferences forward, and remembering podcast interests or exclusions. We should avoid broad autonomous action until memory quality is established.

Chief Technology Officer: We need one shared capability, not a separate memory implementation inside every AI surface. Define the event contract, graph schema, retrieval contract, and policy boundary now. Product teams should integrate through APIs or MCP tools, not direct database queries.

AI Engineering Lead: The workflow is straightforward at a high level: capture an eligible event, resolve entities, write a temporal graph fact, compute or update embeddings, retrieve candidates for the current intent, rerank them, and compose a bounded context pack. The difficult part is deciding what deserves to become memory.

Data Science Lead: We should separate episodic memories from durable preferences. "Played focus music this morning" is an episode. "Prefers low-vocal focus playlists while working" may become a durable preference only after explicit confirmation or repeated supporting evidence. The confidence model needs to preserve that distinction.

Graph Platform Lead: The graph should store facts as time-bounded relationships with provenance. We need valid-from, valid-to, recorded-at, source-event identifiers, confidence, and policy class. Corrections should supersede prior facts; they should not silently overwrite history.

Backend Engineering Lead: Interaction capture must be asynchronous so the user path is not blocked by graph writes. A durable queue can absorb events, and idempotency keys can prevent duplicate writes. Retrieval, however, is synchronous and must have a strict latency budget with cache and no-memory fallback.

MCP Platform Lead: MCP gives us a controlled interface for tools such as `search_memory`, `add_explicit_preference`, `correct_memory`, and `delete_memory`. Each tool needs an explicit schema, authenticated subject scope, rate limits, and an audit event. The model should never receive a generic graph query tool.

Privacy Lead: We must begin with purpose limitation and minimization. Mood patterns can be sensitive or easily misread. We should not store inferred emotional state as a durable profile by default. Consent state, geography, age-related protections, deletion, and retention must be enforced before memory reaches retrieval.

Security Lead: The threat model includes cross-user leakage, prompt injection through stored text, unauthorized tool calls, and replay of stale tokens. Memory text must be treated as untrusted input. Access decisions belong in the service layer, and every read must bind to the authenticated subject.

Data Governance Lead: We also need a data catalog entry for each memory type: source, lawful basis or internal policy basis, allowed purposes, sensitivity, retention period, and deletion path. Graph nodes and vector records must share a stable memory identifier so erasure is complete.

Product Design Lead: Users should not see a technical graph. They need a clear experience: “Spotify remembered this preference,” with the ability to correct or remove it. In some journeys, subtle disclosure is enough. In settings, there should be a consolidated memory-control view.

Quality Engineering Lead: We will create a golden set that includes stable preferences, temporary contexts, contradictions, multilingual phrasing, sparse histories, and adversarial stored content. We need precision at the top of the retrieved set, not just retrieval recall. One wrong memory can be more damaging than three missing ones.

Site Reliability Lead: Our availability target should apply to the memory service, but the AI surface must degrade gracefully. If the graph, embedding service, or MCP server is slow, the experience should proceed without memory and emit a traceable fallback event. We cannot make playback-adjacent journeys depend on a fragile chain.

Analytics Lead: Success needs both offline and online views. Offline: entity resolution accuracy, memory precision, contradiction rate, provenance completeness, and deletion propagation. Online: repeat-query reduction, correction rate, satisfaction, qualified engagement, and retention guardrails.

Operations Lead: The operations console must show ingestion lag, graph-write failures, retrieval latency, top policy rejections, deletion queue status, and experiment cohort allocation. We also need replay tools for failed events without creating duplicates.

Chief Product Officer: The release decision is clear. Ship the smallest governed memory loop that can prove relevance, trust, and operational control. Do not widen the stored-memory taxonomy until the evaluation data supports it.

5. Product Blueprint

5.1 Product Vision

Give Spotify AI experiences a trustworthy continuity layer that understands durable preferences and prior interactions, retrieves only what matters for the current intent, and turns that memory into more relevant responses without weakening user control. The product should make personalization feel coherent across time while keeping memory provenance, policy, correction, and deletion first-class capabilities.

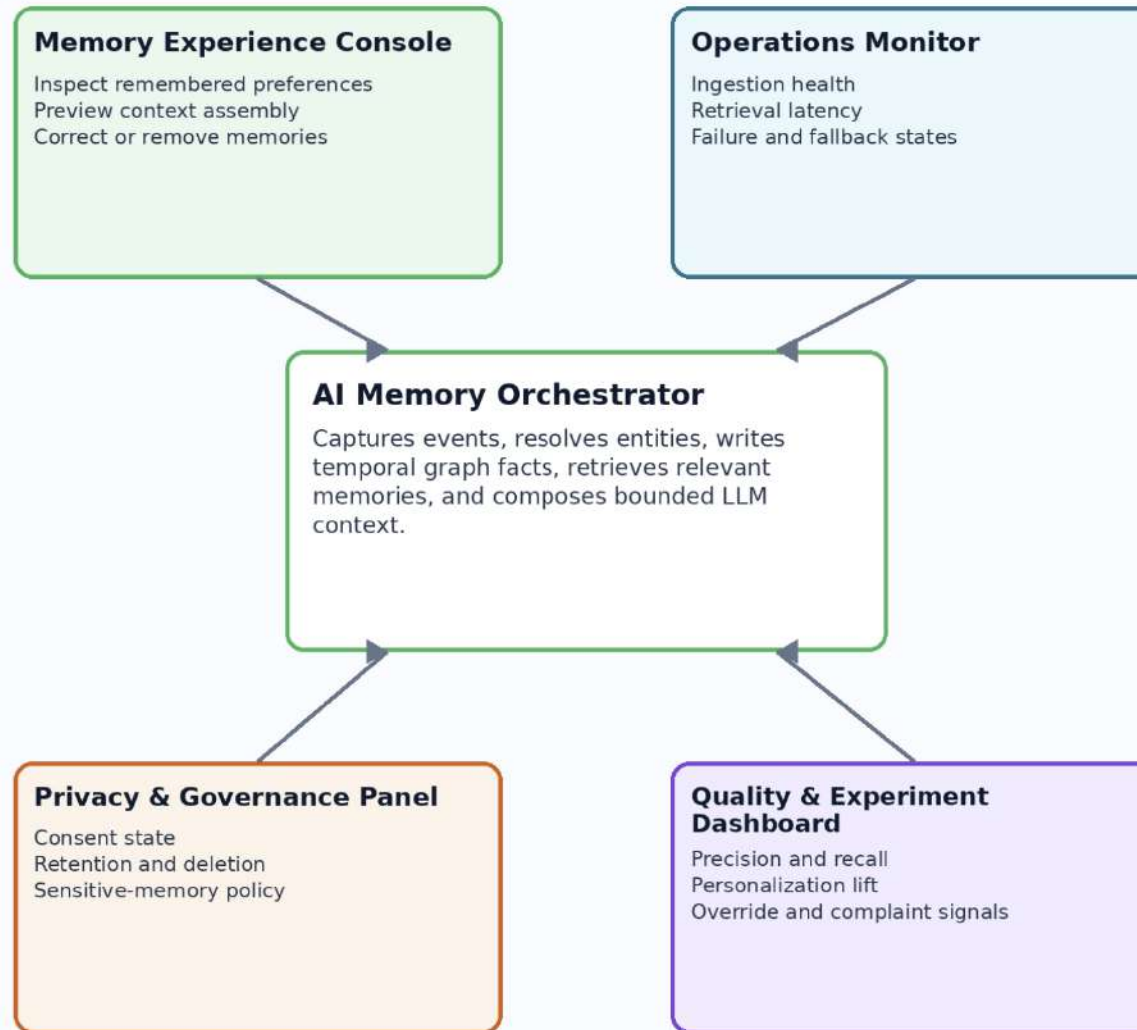
5.2 Product Surfaces

- Memory Experience Console: Internal workspace to inspect a subject-scoped timeline, preview retrieved memories, understand provenance, and validate context composition.
- Memory Control Experience: User-facing controls to review, correct, remove, pause, or opt out of eligible memory behavior.
- Graph and Schema Console: Restricted interface for graph ontology versions, memory types, relationship rules, retention classes, and migration status.
- Retrieval Quality Dashboard: Offline and online views for precision, recall, freshness, contradiction rate, confidence distribution, and experiment outcomes.
- Operations Monitor: Ingestion lag, queue health, graph and vector write failures, retrieval latency, fallbacks, retries, and deletion propagation.
- Policy Review Panel: Memory policy decisions, sensitive-type blocks, consent state, reviewer queues, and audit trace.



Product Surface Map

How Spotify teams govern memory quality and AI personalization



5.3 Primary User Journeys

Journey	Typical User Need	System Behavior	Expected Outcome
Continue a listening conversation	A listener refines music preferences across multiple AI interactions.	Retrieves durable preferences and recent episodes, suppresses stale conflicts, and injects a compact context package.	A response that builds on prior choices without asking the same questions again.
Preserve playlist preferences	A listener consistently asks for a particular energy level, era, language, or exclusion.	Combines explicit statements and repeated evidence, creates a confidence-scored graph relationship, and surfaces it only for relevant requests.	Faster playlist refinement with fewer unwanted recommendations.
Resume podcast discovery	A listener returns to a topic, series, host, or episode style discussed previously.	Traverses topic and content relationships, retrieves recent interactions, and distinguishes completed items from future interests.	A coherent continuation instead of a generic restart.
Correct a memory	A listener states that a stored preference is wrong or no longer applies.	Creates a correction event, closes the prior fact's valid-time, updates indexes, and prioritizes the new explicit statement.	Immediate behavioral change with a traceable correction history.
Remove or pause memory	A listener removes a memory or disables eligible memory use.	Revokes retrieval eligibility, propagates deletion to graph and vector stores, and confirms completion status.	User agency is respected across every connected AI surface.
Investigate a quality issue	A product or operations owner reviews an irrelevant personalized response.	Replays the retrieval trace, shows candidate scores and policy decisions, and links the output to source memories.	Root cause is identified without exposing unrelated subject data.

Memory Lifecycle and Decision Flow

From interaction signal to relevant, governed context

1. Capture interaction

AI conversation, playback action, saved preference, or explicit correction enters with consent and source metadata.



2. Normalize and resolve

Identity scope, entities, timestamps, confidence, and event type are standardized; duplicates are collapsed.



3. Write temporal memory

Durable facts, episodes, and relationships are stored with valid-time, source, and expiry fields.



4. Retrieve for intent

Hybrid graph and embedding retrieval selects memories relevant to the current request and surface policy.



5. Filter and compose

Policy rules remove unsafe, stale, low-confidence, or over-budget memories; the context pack is structured.



6. Generate, explain, learn

The LLM uses the context pack, exposes memory use where appropriate, and records feedback without self-confirming errors.

5.4 Detailed Product Requirements

Interaction Capture and Input Handling

- Accept eligible AI interactions, playback actions, saves, follows, skips, explicit preference statements, and corrections through versioned event contracts.
- Attach subject scope, surface, locale, timestamp, consent state, source-event identifier, and idempotency key to every event.
- Separate raw event retention from memory retention; not every event becomes a retrievable memory.
- Reject malformed, unauthenticated, out-of-policy, or unsupported events before graph processing.

Memory Extraction and Entity Resolution

- Classify events into episodes, explicit preferences, candidate preferences, exclusions, corrections, and non-memory interactions.
- Resolve artists, tracks, albums, playlists, shows, episodes, topics, activities, and contextual concepts to canonical identifiers.
- Deduplicate semantically equivalent statements while retaining source lineage.
- Assign confidence and policy class using deterministic rules plus structured model output.

Temporal Graph Memory Layer

- Represent memory facts as versioned nodes and relationships with valid-from, valid-to, recorded-at, source, confidence, and status.
- Support contradiction, supersession, expiry, and explicit correction without erasing audit history prematurely.
- Maintain subject isolation at the query boundary and prevent traversal across unauthorized identity scopes.
- Use graph constraints and idempotent upserts to protect entity and relationship integrity.

Embeddings and Retrieval

- Generate embeddings only for approved memory fields and store vectors under the same stable memory identifier used in the graph.
- Use hybrid candidate generation: graph traversal for relational relevance and vector similarity for semantic relevance.
- Rerank by current intent, explicitness, confidence, recency, repetition, surface policy, and negative feedback.
- Apply diversity and context-budget limits so one preference or content cluster does not dominate the context pack.

MCP and Tool Interface

- Expose narrow tools for search, explicit preference write, correction, deletion, and memory explanation.
- Require typed input/output schemas, authenticated subject binding, authorization checks, rate limits, and audit events.
- Return structured provenance and policy metadata; never expose unrestricted graph query capability to the model.
- Version tool contracts and preserve backward compatibility for integrated AI surfaces.

Context Composition and LLM Integration

- Assemble a structured context package with memory identifier, fact, type, confidence, time, source class, and relevance reason.
- Exclude memories that are expired, contradicted, disallowed, low-confidence, or outside the active surface policy.
- Treat stored free text as untrusted data and isolate it from system instructions to reduce prompt-injection risk.
- Provide a deterministic no-memory fallback and record which memories influenced each response.

User Control, Privacy, and Safety

- Provide review, correction, deletion, pause, and opt-out paths with clear state and propagation status.
- Block or tightly govern sensitive inferred attributes; do not convert transient emotional cues into durable memory by default.
- Apply retention by memory type, geography, age-related policy, consent state, and legal or internal governance requirements.
- Support subject-access and deletion workflows across graph, vector, cache, operational metadata, and backups according to policy.

Observability and Operations

- Trace event intake, transformation, graph write, embedding write, candidate retrieval, reranking, policy filtering, and context injection.
- Monitor ingestion lag, write failures, retrieval latency, cache effectiveness, fallback rate, deletion backlog, and policy rejection rate.
- Provide dead-letter handling and idempotent replay for recoverable ingestion failures.
- Redact sensitive payloads from logs while preserving identifiers needed for investigation.

Experimentation and Quality Review

- Support memory-enabled and memory-disabled experiments under consistent cohort allocation and guardrails.
- Maintain golden sets for explicit preference, temporal change, contradiction, multilingual interaction, sparse history, and malicious stored text.
- Capture user corrections and reviewer decisions without automatically reinforcing model-generated claims.
- Gate taxonomy expansion on precision, safety, and deletion-propagation thresholds.

5.5 Non-Functional Expectations

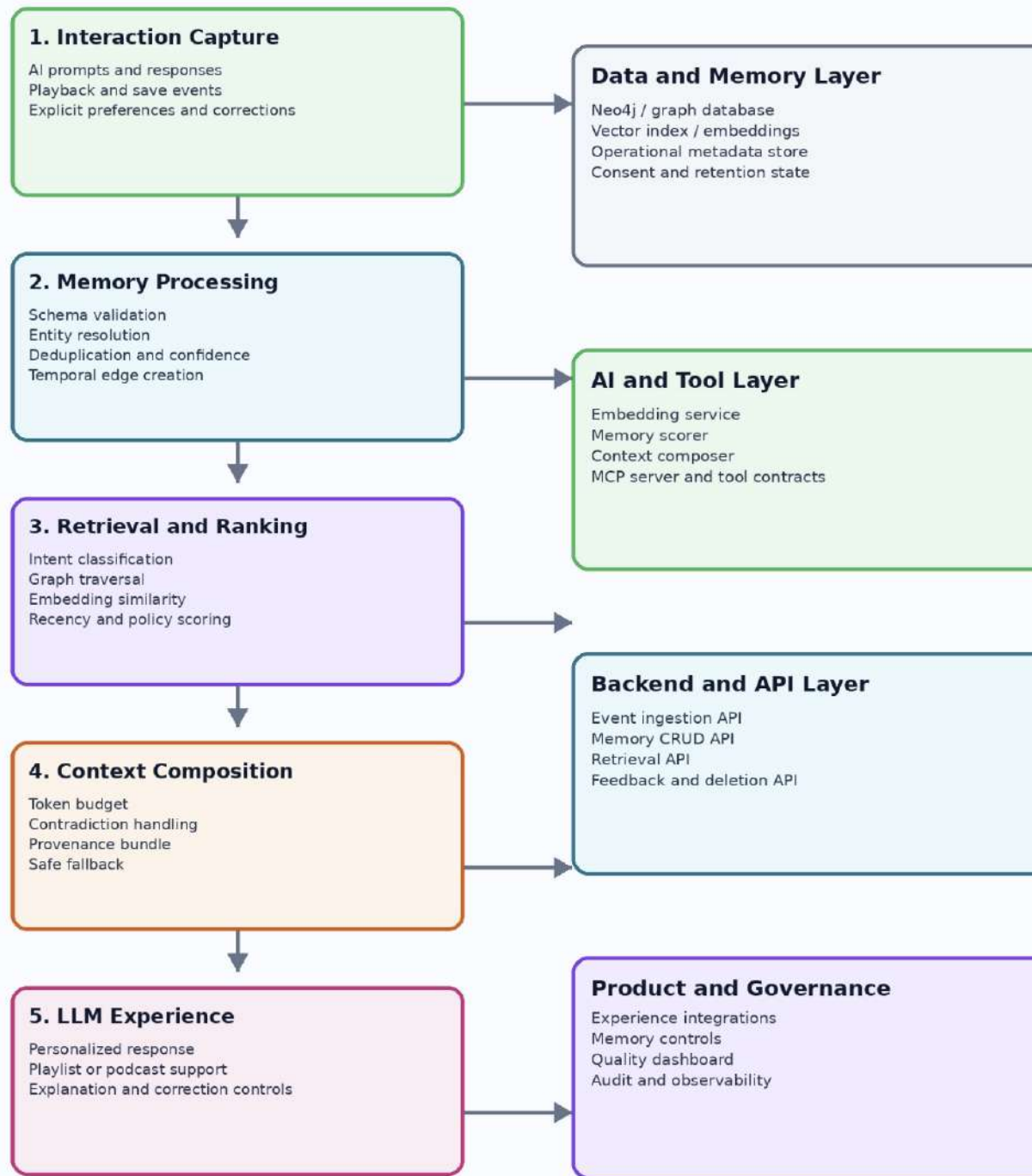
Area	Expectation	Notes
Reliability	Memory retrieval must fail open to a non-personalized response.	No partial or cross-subject context may be injected after a timeout or authorization failure.
Performance	P95 retrieval and context composition should remain within a 250 ms service budget for the pilot.	Use bounded candidate sets, indexed graph queries, vector search, and short-lived subject-scoped caches.
Security	Every read and write must bind to authenticated subject and service identities.	Encrypt in transit and at rest; restrict tools; validate stored text; rotate secrets; audit privileged access.
Privacy	Collection and retrieval must be purpose-limited, minimal, and reversible.	Consent, sensitivity, retention, geography, age-related protections, deletion, and export are policy inputs.
Maintainability	Memory schemas, scoring policies, MCP tools, and prompts must be versioned and independently testable.	Migrations require compatibility checks and rollback plans.
Usability	Internal operators and eligible users must understand what was remembered and how to correct it.	Use plain-language provenance and avoid exposing raw graph internals.
Scalability	Partition workloads and indexes so growth in event volume does not degrade retrieval isolation or latency.	Asynchronous writes, idempotency, queue backpressure, and capacity tests are required.
Observability	Each response must link to a trace across retrieval, filtering, context composition, and fallback decisions.	Dashboards and alerts must expose SLOs without logging raw sensitive content.
Deployment readiness	Every service must have health checks, environment configuration, migration scripts, rollback, and runbooks.	A controlled review URL and realistic seeded data set must be available for leadership validation.

6. Technical Architecture

The architecture uses asynchronous capture for durability and synchronous retrieval for experience-time relevance. Interaction events enter a governed processing pipeline, become time-bounded graph facts and vector representations, and are retrieved through a hybrid ranking path. MCP and service APIs mediate all product access. The LLM receives a compact, policy-filtered context package rather than direct database access.

Technical Architecture

Temporal graph memory with embedding retrieval and MCP-mediated access



6.1 Architecture Narrative

- 1. Capture and authorize.** An AI surface emits a versioned interaction event. The ingestion API verifies service identity, subject scope, consent state, schema version, timestamp, and idempotency key. Accepted events enter a durable queue so graph processing does not block the experience.
- 2. Extract and normalize.** A memory processor classifies the event, extracts candidate facts into a typed schema, resolves canonical content and concept entities, and applies minimization and sensitivity rules. Deterministic validators reject unsupported or unsafe fields.
- 3. Write temporal graph.** Approved facts are upserted into a graph database as entities and time-bounded relationships. Provenance, confidence, policy class, valid-time, and expiry remain attached. Contradictions close or supersede prior facts instead of silently replacing history.
- 4. Create embeddings.** Approved semantic fields are embedded and written to a vector index under the graph memory identifier. Deletion and update events use the same identifier to keep graph and vector state aligned.
- 5. Retrieve candidates.** At response time, the retrieval API classifies current intent, requests relational candidates from the graph, semantic candidates from the vector index, and recent explicit signals from operational storage. Candidate generation is subject-scoped and bounded.
- 6. Rerank and govern.** A scorer combines intent fit, explicitness, confidence, recency, repetition, correction status, surface policy, and negative feedback. The policy engine removes prohibited, expired, contradicted, or over-budget items.
- 7. Compose context.** The context composer returns a typed memory package with provenance and relevance reasons. Stored text is quoted as data, not instructions. If confidence or system health is insufficient, the composer returns an explicit no-memory result.
- 8. Generate and observe.** The LLM uses the memory package to produce a personalized response. The orchestration layer records retrieval identifiers, policy decisions, latency, fallback state, and feedback links while excluding unnecessary content from logs.

6.2 Recommended Technology Choices

Layer	Preferred Options	Why It Fits
Product interface	Next.js, React, Tailwind CSS	Supports internal consoles, memory controls, trace views, and a deployable product shell.
Backend/API	FastAPI with Pydantic, or TypeScript services	Typed contracts, asynchronous endpoints, validation, and clear separation between product and data layers.
Event transport	Apache Kafka or Redpanda	Durable event capture, partitioning, replay, backpressure, and consumer isolation.
Graph memory	Neo4j with Graphiti patterns	Temporal graph relationships, indexed traversal, provenance, and evolving user-content relationships.
Embeddings	SentenceTransformers or approved embedding service	Encodes semantic similarity for preference and episode retrieval; can be benchmarked and versioned.
Vector retrieval	Neo4j vector indexes, Qdrant, or pgvector	Supports bounded semantic candidate retrieval linked to stable memory identifiers.
Operational store	PostgreSQL	Stores consent state, ingestion status, tool audit, experiments, feedback, and deletion jobs.
Cache	Redis	Short-lived subject-scoped caching, rate limiting, and idempotency support.
Tool interface	Model Context Protocol SDK	Typed, narrow memory tools with explicit authorization and audit boundaries.
Observability	OpenTelemetry, Prometheus, Grafana	Distributed traces, service metrics, SLO monitoring, and incident investigation.
Deployment	Docker, Kubernetes or managed containers; Vercel for internal web shell where appropriate	Separates stateless services from persistent stores and supports controlled releases and rollback.

6.3 Data Assets Required

- Versioned interaction-event schema with subject scope, surface, locale, timestamp, event type, consent state, source identifier, and idempotency key.
- Canonical catalog entity data for artists, tracks, albums, playlists, shows, episodes, topics, and supported contextual concepts.
- Temporal memory graph schema covering episodes, explicit preferences, candidate preferences, exclusions, corrections, supersession, expiry, and provenance.
- Embedding corpus and metadata linking every vector to a memory identifier, embedding model version, approved text fields, and deletion status.
- Policy registry describing allowed memory types, sensitivity, purposes, retention, geography, age-related handling, and retrieval eligibility.
- Golden evaluation cases with expected memory extraction, graph state, retrieved candidates, policy outcome, and context package.
- Operational telemetry for ingestion, write success, retrieval latency, fallback, context size, correction, deletion, and experiment cohort allocation.
- Seeded demonstration data that is synthetic or explicitly approved and contains no real private interaction history.

6.4 Deployment Flow

- 1. Repository and environments.** Maintain a protected GitHub repository with trunk-based or short-lived branch workflows, required checks, dependency scanning, and separate local, development, staging, and production configuration.
- 2. Frontend deployment.** Deploy the internal review interface through Vercel or Netlify where permitted, or through the company's standard internal web platform. The deployed URL must use authenticated access for any non-synthetic data.
- 3. Backend and MCP deployment.** Package APIs, workers, policy services, and MCP server as separate containers. Deploy behind an API gateway with workload identity, rate limits, private networking, and health probes.
- 4. Graph, vector, and operational stores.** Provision isolated development and staging instances. Apply schema constraints and migrations before traffic, verify backups, and test deletion propagation across all stores.
- 5. Environment variables and secrets.** Use a secret manager for model credentials, database endpoints, signing keys, encryption material, tracing endpoints, and feature flags. Commit only a safe `.env.example`.
- 6. Release and rollback.** Run schema compatibility checks, offline quality gates, security tests, synthetic smoke tests, canary release, and automatic rollback on SLO breach.
- 7. Review readiness.** Publish the controlled application URL in README, document setup and architecture, seed approved demo scenarios, and provide a recorded end-to-end walkthrough.

7. Detailed Solution Plan and Build Approach

7.1 Repository and Folder Structure

spotify-personalized-ai-memory-system/

|-- apps/

||-- memory-console/ # Next.js internal product surface

|`-- memory-controls/ # Review, correction, deletion UI

|-- services/

||-- ingestion-api/ # Event authentication and validation

||-- memory-processor/ # Extraction, entity resolution, policy

||-- retrieval-api/ # Hybrid retrieval and ranking

||-- context-composer/ # Bounded LLM context packages

||-- memory-mcp-server/ # Narrow, typed memory tools

|`-- deletion-orchestrator/ # Cross-store erasure workflow

|-- packages/

||-- contracts/ # Events, APIs, MCP, structured outputs

||-- graph-schema/ # Nodes, relationships, constraints

||-- policy-engine/ # Eligibility and retention rules

||-- evaluation/ # Golden sets and scorers

```
|-- infrastructure/

|-- containers/

|-- kubernetes/

|-- database-migrations/

|-- monitoring/

|-- data/

|-- synthetic/

|-- schemas/

|-- golden-sets/

|-- tests/

|-- unit/

|-- integration/

|-- contract/

|-- security/

|-- end-to-end/

|-- docs/

|-- architecture/

|-- api/

|-- privacy-and-security/

|-- runbooks/
```

|-- .env.example

|-- docker-compose.yml

`-- README.md

7.2 Data and Knowledge Preparation Approach

- 1. Define the memory taxonomy.** Start with explicit preference, exclusion, correction, episodic interaction, and approved inferred-preference types. Document examples, counterexamples, sensitivity, retention, and retrieval eligibility for each type.
- 2. Create event and memory contracts.** Use JSON Schema or Pydantic models for interaction events, extracted candidates, graph writes, vectors, retrieval candidates, policy decisions, and context packages. Version every contract.
- 3. Prepare synthetic histories.** Create realistic multi-session histories that include stable tastes, temporary contexts, evolving interests, contradictions, multilingual statements, and opt-out or deletion events.
- 4. Normalize and resolve entities.** Map catalog entities to stable identifiers; normalize locale and timestamps; maintain alias tables; define confidence thresholds for ambiguous topic or activity concepts.
- 5. Build the graph safely.** Apply uniqueness constraints, subject partition keys, indexed temporal fields, and idempotent write semantics. Preserve provenance and never treat LLM extraction as authoritative without validation.
- 6. Align graph and vectors.** Generate vectors only after policy approval. Store graph memory ID, model version, text-field version, and deletion status with each vector so updates and erasure remain deterministic.
- 7. Establish evaluation truth.** For every golden history, specify the expected graph state, retrieved top memories, prohibited memories, context budget, and correction or deletion outcome.

7.3 Backend and API Approach

API	Purpose
POST /v1/events	Accept an eligible interaction event; validate subject, consent, schema, idempotency, and source.
POST /v1/memories/extract	Convert an approved event into typed candidate memories for deterministic validation.
POST /v1/memories	Create an explicit or approved memory and return stable ID, graph version, and policy state.
POST /v1/memories/search	Return ranked subject-scoped memories for intent, surface, locale, and token budget.
POST /v1/context/compose	Apply policy and build the context package consumed by an AI orchestrator.
PATCH /v1/memories/{memory_id}	Correct, supersede, expire, or change an eligible memory under optimistic concurrency.
DELETE /v1/memories/{memory_id}	Start cross-store deletion and return a traceable job identifier.
GET /v1/deletions/{job_id}	Report graph, vector, cache, operational-store, and backup-policy status.
POST /v1/feedback	Record relevance, correction, rejection, or experience feedback without self-validating model output.
GET /v1/traces/{trace_id}	Return authorized retrieval and policy decisions with sensitive fields redacted.

Request flow: the gateway authenticates workload and subject scope, validates a typed payload, attaches a correlation identifier, executes service logic, and returns a structured result with policy and trace metadata. Writes are idempotent; asynchronous work returns a job state. Errors use stable codes for validation, authorization, policy denial, conflict, dependency timeout, and retryable service failure. Logs contain identifiers and outcomes, not raw private content.

7.4 AI Workflow Approach

- 1. Capture user interactions.** Receive eligible interactions and product signals with source, subject, consent, locale, timestamp, and idempotency metadata. Keep the user path independent of downstream graph-write latency.
- 2. Store in graph.** Extract typed candidate memories, resolve entities, apply policy, and write time-bounded facts and episodes to the graph. Create or update embeddings only after approval.
- 3. Retrieve relevant memory.** Classify current intent; fetch graph and semantic candidates; add explicit recent signals; rerank by relevance, explicitness, confidence, recency, correction state, and surface policy.
- 4. Inject into LLM context.** Compose a small structured memory block with provenance and relevance reasons. Quote memory as data, enforce the token budget, and return no-memory when quality or service health is insufficient.
- 5. Generate and capture feedback.** Generate the personalized response, link it to memory IDs and policy decisions, and accept explicit corrections or experience feedback through governed update paths.

7.5 Prompting, Reasoning, and Agentic Approach

The system should use structured prompts at narrow decision points rather than one unconstrained agent. Memory extraction receives an event plus an allowed taxonomy and returns typed candidates. Entity resolution receives candidates plus canonical search results. Retrieval scoring is primarily deterministic; an LLM reranker may be tested only on a bounded candidate set with a structured score and reason. Context composition is deterministic after policy filtering.

- System instruction: define the role, allowed memory taxonomy, prohibited inferences, subject boundary, temporal rules, and requirement to return “no memory” when evidence is insufficient.
- Input block: include current intent, surface policy, locale, candidate memories, provenance, timestamps, confidence, correction state, and strict context budget.
- Structured output: use fields such as `memory_id`, `decision`, `normalized_fact`, `entities`, `relevance_score`, `confidence`, `temporal_scope`, `policy_flags`, and `reason`.
- MCP tools: expose `search_memory`, `add_explicit_preference`, `correct_memory`, `delete_memory`, and `explain_memory_use`. Require server-side authorization and validation for every call.
- Validation: reject unknown entities, unsupported claims, invented memory IDs, invalid timestamps, policy-ineligible types, and context items without provenance.
- Human review: require policy or data-governance approval for new memory types, sensitive categories, and scoring-rule changes; routine user corrections should not require manual approval.
- Memory safety: never write a new durable fact solely because the model generated it in a response. Only eligible user or product evidence can create memory.

7.6 Frontend Product Approach

- Overview: Service health, ingestion lag, retrieval SLO, fallback rate, quality metrics, experiment status, and deletion backlog.
- Subject-scoped memory explorer: Search only with approved support or test identities; show timeline, graph relationships, source type, confidence, and status.
- Context preview: Enter a current intent and surface; display candidate retrieval, ranking, policy removals, final context pack, and token usage.
- Correction and deletion: Correct or remove eligible memories, show propagation status, and prevent silent partial completion.
- Schema and policy view: Read-only view for most roles; version history, allowed fields, retention, sensitivity, and rollout state.
- Quality review: Golden-set runs, failure clusters, multilingual cases, contradiction cases, and side-by-side memory-enabled comparisons.
- Audit trace: Authorized view of tool calls, service decisions, memory identifiers, timestamps, and redacted outcomes.

7.7 Testing Approach

Test Area	Coverage
Functional	Event validation, extraction, graph upsert, vector alignment, hybrid retrieval, context composition, correction, expiry, deletion, and feedback.
Contract	Backward compatibility for event, API, MCP tool, graph, vector, and context-package schemas.
AI output	Typed-output validity, unsupported-memory rejection, entity grounding, top-k precision, contradiction handling, and no-memory decisions.
Security	Cross-subject isolation, authorization bypass attempts, prompt injection through stored content, tool abuse, secret leakage, replay, and rate limits.
Privacy	Consent changes, retention expiry, sensitive-type rejection, export, deletion propagation, cache invalidation, and backup-policy evidence.
Performance	Ingestion throughput, queue backpressure, graph and vector scale, P50/P95/P99 retrieval, cold cache, hot partitions, and dependency timeouts.
Resilience	Graph outage, vector outage, embedding timeout, partial write, duplicate event, stale index, malformed model output, and rollback.
Experience	Repeated refinement, changed preference, temporary context, playlist exclusion, podcast continuation, sparse history, opt-out, and correction.

8. Reference Stack and Useful Links

Category	Tool / Resource	Link	Why It Is Useful
Provided GitHub Reference	Graphiti	Open resource	Starting point for graph-memory and MCP integration patterns.
Provided GitHub Reference	Context	Open resource	Starting point for context assembly and workflow design.
Official Documentation	Model Context Protocol	Open resource	Protocol concepts and SDK guidance for typed tool access.
Official Documentation	Neo4j Python Driver	Open resource	Supported Python access patterns, transactions, and query execution.
Official Documentation	Neo4j Vector Indexes	Open resource	Vector index creation and query concepts alongside graph data.
Official Documentation	Qdrant	Open resource	Open-source vector retrieval, filtering, and operations.
Official Documentation	FastAPI	Open resource	Typed Python APIs, validation, dependency injection, and OpenAPI.
Official Documentation	Pydantic	Open resource	Structured event, tool, and model-output validation.
Official Documentation	Next.js	Open resource	Web product and server integration documentation.
Official Documentation	Vercel	Open resource	Frontend and controlled preview deployment guidance.
Official Documentation	OpenTelemetry	Open resource	Distributed traces, metrics, and log correlation.
Official Documentation	Prometheus	Open resource	Service metrics and alerting foundations.
GitHub Repository	Graphiti	Open resource	Open-source temporal knowledge-graph memory reference.
GitHub Repository	pgvector	Open resource	Vector similarity search inside PostgreSQL.
GitHub Repository	SentenceTransformers	Open resource	Open-source embedding and reranking reference.
GitHub Repository	MCP Python SDK	Open resource	Python server and client implementation patterns.
GitHub Repository	MCP TypeScript SDK	Open resource	TypeScript server and client implementation patterns.

9. Evaluation Metrics

Category	Marks	What Reviewers Will Look For
Product understanding and business framing	8	Clear articulation of continuity, personalization, retention value, user agency, and constrained first-release scope.
Product experience	10	Usable memory controls, context preview, operations views, correction and deletion journeys, and understandable provenance.
Backend and API implementation	12	Typed contracts, authentication, idempotency, queue integration, error handling, stable APIs, and audit events.
AI workflow implementation	15	Accurate extraction, entity resolution, retrieval, reranking, policy filtering, context composition, and structured outputs.
Graph, embeddings, and memory quality	15	Temporal graph integrity, provenance, vector alignment, top-k precision, contradiction handling, correction, and expiry.
Reliability, privacy, and safety	14	Subject isolation, safe fallback, consent, sensitive-memory controls, deletion propagation, prompt-injection defense, and SLOs.
Deployment readiness	8	Reproducible environments, migrations, secrets, health checks, rollback, controlled URL, and operational dashboards.
Documentation quality	6	README, architecture, API and MCP contracts, schema, policy, runbooks, screenshots, and setup clarity.
Demo quality	6	Credible walkthrough of capture, graph write, retrieval, personalization, correction, deletion, and fallback.
Team collaboration	6	Clear ownership and contribution evidence across product, AI, data, backend, frontend, privacy, security, quality, and operations.

Scoring rule

Total available marks: 100. There are no bonus marks. Any critical privacy, security, cross-subject isolation, or deletion failure blocks release regardless of aggregate score.

10. Final Deliverables

- Complete GitHub repository link containing the codebase, infrastructure definitions, tests, schemas, prompts, policy rules, and technical documentation.
- Deployed application link clearly stated in README and protected appropriately for any non-synthetic environment.
- Working end-to-end demo video uploaded to Google Drive with "Anyone with the link can view" access.
- README covering setup, architecture, screenshots, product explanation, APIs, MCP tools, environment variables, data model, limitations, and team contribution.
- Required synthetic data, graph schema, vector metadata schema, event contracts, MCP tool definitions, golden evaluation set, workflow files, and runbooks.
- Release evidence covering quality results, security and privacy checks, latency, resilience, deletion propagation, known risks, and rollback readiness.

11. Final Submission Checklist

- ☐ GitHub repository link is accessible to authorized reviewers.
- ☐ Complete codebase and infrastructure definitions are available.
- ☐ README is complete and includes setup, architecture, screenshots, APIs, MCP tools, limitations, and contribution ownership.
- ☐ Deployed application link is added inside README.
- ☐ Deployed application is working and protected for its data classification.
- ☐ Core workflow works end to end: capture → graph write → retrieval → context injection → response → feedback.
- ☐ AI workflow is implemented with typed outputs, validation, and safe fallback.
- ☐ Graph, embedding, retrieval, and memory layers are aligned and observable.
- ☐ Consent, correction, retention, opt-out, and deletion flows are working.
- ☐ Cross-subject isolation and stored-content prompt-injection tests pass.
- ☐ References, API contracts, graph schema, MCP definitions, and operational documentation are included.
- ☐ Demo video is accessible with anyone-view access.

12. Closing Note

The Personalized AI Memory System is a shared product capability for continuity, not a general-purpose archive of user behavior. Its value depends on disciplined memory selection, temporal graph integrity, high-precision retrieval, bounded context composition, and complete user control. A strong first release will prove that Spotify can make AI experiences feel meaningfully more personal while making every remembered fact traceable, correctable, removable, and safe to use.

Decision standard

Expand only when memory relevance, privacy controls, deletion propagation, latency, and user outcomes meet the agreed release gates. The architecture should make the safe path the default path.
